

Experiment No. 1

Implementation of Fundamental Image Processing Operations using Python and OpenCV

Aim

To develop a comprehensive understanding of fundamental image processing techniques by implementing image acquisition, storage, color space conversion, geometric transformations, and image enhancement operations using Python and OpenCV.

Purpose

Image processing forms the core of modern computer vision systems used in autonomous driving, healthcare, surveillance, robotics, and industrial automation. This experiment demonstrates how digital images are read, stored, converted, resized, transformed, enhanced, and cropped before advanced computer vision tasks.

Prerequisites

- Basic knowledge of Python
- Knowledge of pixels, resolution, and color channels
- OpenCV, NumPy, and Matplotlib
- Google Colab / Jupyter Notebook

```
!pip install opencv-python-headless -q

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
import shutil
from google.colab import files

print("Libraries imported successfully.")

Libraries imported successfully.
```

1. Image Acquisition

```
# Upload an image from your computer
uploaded = files.upload()

image_name = list(uploaded.keys())[0]
image = cv2.imread(image_name)
```

```
if image is None:
    raise ValueError("The uploaded file is not a valid image.")

print("Image loaded successfully!")
print("File name:", image_name)

<IPython.core.display.HTML object>

Saving 9011064.webp to 9011064.webp
Image loaded successfully!
File name: 9011064.webp
```

2. Display the Original Image

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Original Image



3. Image Properties

```

height, width, channels = image.shape

print("Image Properties")
print("-----")
print("Width:", width, "pixels")
print("Height:", height, "pixels")
print("Resolution:", width, "x", height)
print("Number of Channels:", channels)
print("Data Type:", image.dtype)
print("Total Pixels:", width * height)

```

Image Properties

```

-----
Width: 3840 pixels
Height: 2160 pixels
Resolution: 3840 x 2160
Number of Channels: 3
Data Type: uint8
Total Pixels: 8294400

```

4. Save Image in JPEG and PNG Formats

```

cv2.imwrite("output_image.jpg", image)
cv2.imwrite("output_image.png", image)

jpeg_size = os.path.getsize("output_image.jpg")
png_size = os.path.getsize("output_image.png")

print("Images saved successfully.")
print("JPEG size:", jpeg_size, "bytes")
print("PNG size:", png_size, "bytes")

```

```

Images saved successfully.
JPEG size: 849244 bytes
PNG size: 5154203 bytes

```

5. Color Space Conversion

```

gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lab_image = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
plt.imshow(image_rgb)
plt.title("Original RGB Image")
plt.axis("off")

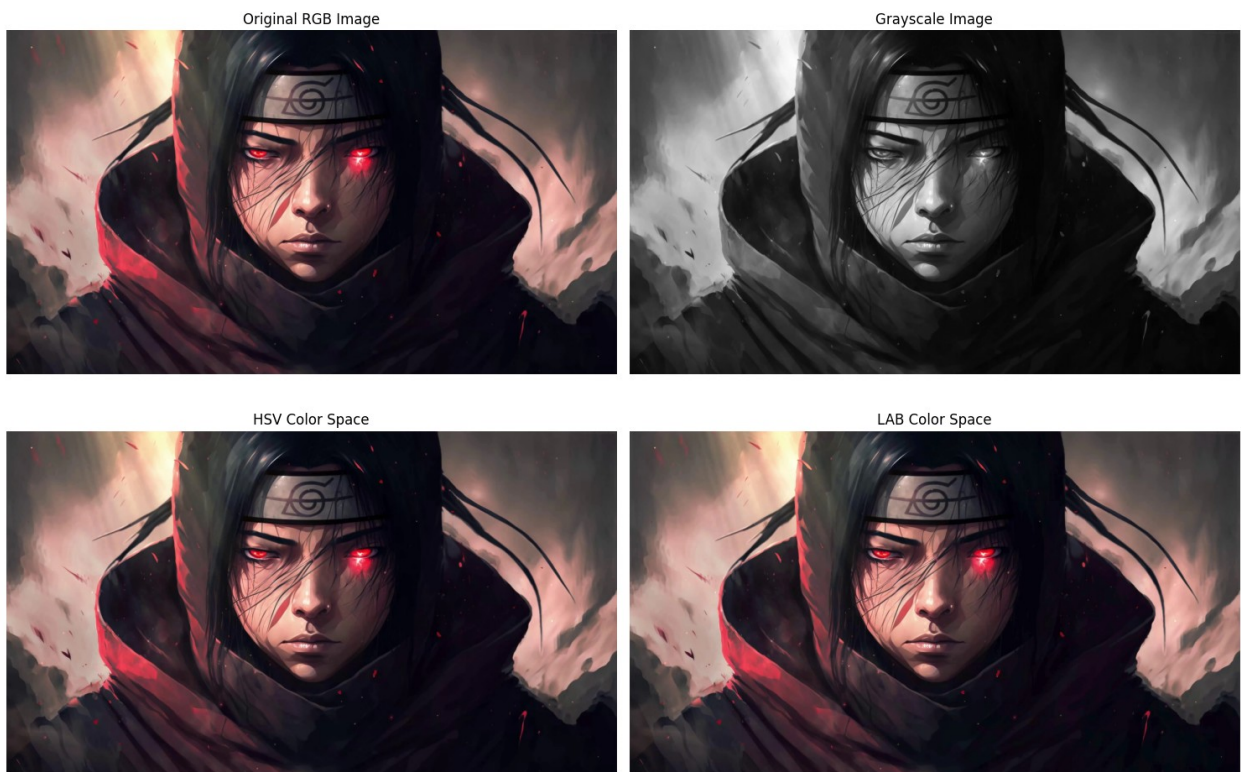
```

```
plt.subplot(2, 2, 2)
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(hsv_image, cv2.COLOR_HSV2RGB))
plt.title("HSV Color Space")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(lab_image, cv2.COLOR_LAB2RGB))
plt.title("LAB Color Space")
plt.axis("off")

plt.tight_layout()
plt.show()
```



6. Geometric Transformations

```
# Resizing
resized_image = cv2.resize(image, (300, 300))

# Rotation
```

```
rotated_image = cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE)

# Horizontal flip
horizontal_flip = cv2.flip(image, 1)

# Vertical flip
vertical_flip = cv2.flip(image, 0)

plt.figure(figsize=(16, 12))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB))
plt.title("Resized - 300 x 300")
plt.axis("off")

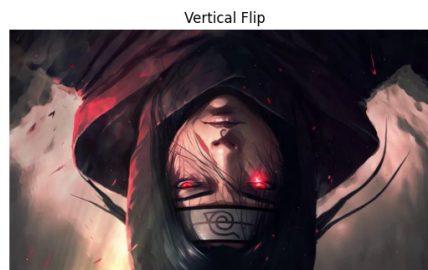
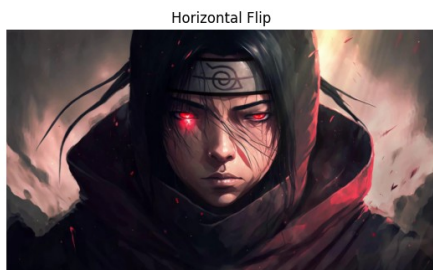
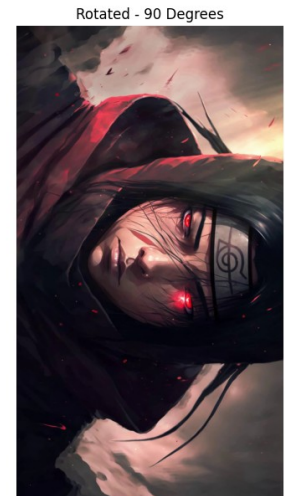
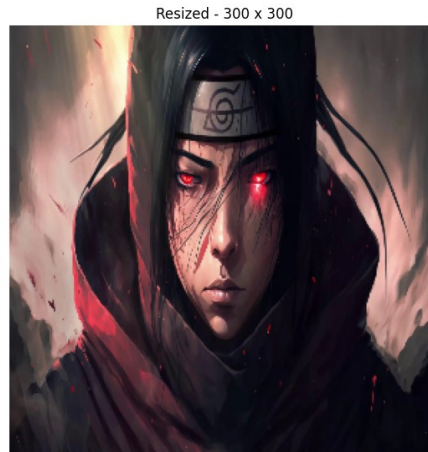
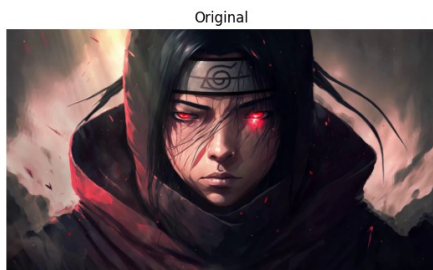
plt.subplot(2, 3, 3)
plt.imshow(cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB))
plt.title("Rotated - 90 Degrees")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
plt.title("Vertical Flip")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Original size:", width, "x", height)
print("Resized size:", resized_image.shape[1], "x",
      resized_image.shape[0])
```



Original size: 3840 x 2160
Resized size: 300 x 300

7. Image Complement / Negative

```
negative_image = cv2.bitwise_not(image)

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(negative_image, cv2.COLOR_BGR2RGB))
plt.title("Complement / Negative Image")
plt.axis("off")
plt.show()

print("Negative image generated successfully.")
print("Formula: Negative Pixel = 255 - Original Pixel")
```

Complement / Negative Image



Negative image generated successfully.
Formula: Negative Pixel = 255 - Original Pixel

8. Region of Interest (ROI)

```
h, w = image.shape[:2]

# Central 50% region
x1 = int(w * 0.25)
y1 = int(h * 0.25)
x2 = int(w * 0.75)
y2 = int(h * 0.75)

roi = image[y1:y2, x1:x2]

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Region of Interest (ROI)")
plt.axis("off")
plt.show()

print("ROI Coordinates:")
print("x1 =", x1, "y1 =", y1)
print("x2 =", x2, "y2 =", y2)
print("ROI size:", roi.shape[1], "x", roi.shape[0])
```

Region of Interest (ROI)



```
ROI Coordinates:  
x1 = 960 y1 = 540  
x2 = 2880 y2 = 1620  
ROI size: 1920 x 1080
```

9. Save All Processed Images

```
cv2.imwrite("grayscale_image.jpg", gray_image)  
cv2.imwrite("resized_image.jpg", resized_image)  
cv2.imwrite("rotated_image.jpg", rotated_image)  
cv2.imwrite("horizontal_flip.jpg", horizontal_flip)  
cv2.imwrite("vertical_flip.jpg", vertical_flip)  
cv2.imwrite("negative_image.jpg", negative_image)  
cv2.imwrite("roi_image.jpg", roi)
```

```
print("All processed images saved successfully.")
```

All processed images saved successfully.

Observations

1. The image was successfully acquired and displayed.
2. Image dimensions, resolution, channels, and data type were determined.
3. The image was saved in JPEG and PNG formats.
4. The image was converted into grayscale, HSV, and LAB color spaces.

5. Grayscale conversion reduced the image to one intensity channel.
6. Resizing changed the spatial dimensions.
7. Rotation changed the orientation by 90 degrees.
8. Horizontal and vertical flipping produced mirror images.
9. The complement operation inverted pixel intensities.
10. A central Region of Interest was successfully extracted.

1. What is a digital image? Differentiate between grayscale and color images.

Answer: A digital image is a two-dimensional representation of a visual scene stored as pixels. A grayscale image generally has one intensity channel, while a color image generally has three channels such as red, green, and blue. Grayscale images require less storage and are simpler to process, whereas color images contain complete color information.

2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

Answer: RGB uses Red, Green, and Blue channels. BGR uses Blue, Green, and Red and is the common channel order used by OpenCV. HSV uses Hue, Saturation, and Value and is useful for color detection. LAB uses Lightness, A, and B components and is useful for color analysis and processing.

3. What is the purpose of converting an image to grayscale before further processing?

Answer: Grayscale conversion reduces a color image to one intensity channel. It reduces computational complexity and memory requirements and is useful for operations such as edge detection, thresholding, contours, and many feature extraction techniques.

4. Explain image complement (negative image) and its practical applications.

Answer: An image complement is produced by inverting each pixel intensity. For an 8-bit image, $\text{Negative Pixel} = 255 - \text{Original Pixel}$. It can be used for visual enhancement, medical image analysis, document processing, and photographic effects.

5. Differentiate between image resizing, cropping, and scaling.

Answer: Resizing changes the dimensions of an image. Cropping removes unwanted areas and keeps a selected region. Scaling enlarges or reduces an image using a scale factor.

6. What is a Region of Interest (ROI), and why is it important in computer vision?

Answer: ROI is a selected portion of an image containing the area of interest. Processing only the ROI reduces unnecessary computation and allows computer vision systems to focus on relevant information.

7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

Answer: OpenCV is open source and provides a large collection of optimized functions for image processing, video processing, transformations, feature extraction, object detection, and real-time computer vision. It also works efficiently with NumPy.

8. Explain how image resolution and pixel intensity influence image quality.

Answer: Resolution indicates the number of pixels used to represent an image. Higher resolution generally provides more spatial detail. Pixel intensity represents brightness; in an 8-bit grayscale image, values normally range from 0 to 255. Suitable resolution, intensity, and contrast improve visual information.

9. Mention five real-world applications where basic image preprocessing is an essential step.

Answer: Five applications are medical imaging, autonomous vehicles, face recognition, surveillance systems, and industrial quality inspection. Preprocessing is also important in robotics, OCR, satellite imaging, and agricultural monitoring.

10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Answer: Preprocessing makes input images more consistent and can remove irrelevant information. Resizing, normalization, color conversion, noise reduction, contrast enhancement, and ROI extraction can make useful patterns easier to learn and reduce computational requirements.

Conclusion

The experiment successfully demonstrated fundamental image processing operations using Python, OpenCV, NumPy, and Matplotlib. Image acquisition, storage, color space conversion,

resizing, rotation, flipping, image complement, and ROI extraction were implemented successfully.

These preprocessing techniques form an important foundation for advanced computer vision applications such as object detection, image classification, facial recognition, medical image analysis, robotics, and autonomous systems.

Result

The fundamental image processing operations were successfully implemented using Python and OpenCV. The processed images and their properties were obtained successfully, demonstrating the importance of image preprocessing as a basic stage in computer vision applications.

```
import os
import shutil

output_files = [
    "output_image.jpg",
    "output_image.png",
    "grayscale_image.jpg",
    "resized_image.jpg",
    "rotated_image.jpg",
    "horizontal_flip.jpg",
    "vertical_flip.jpg",
    "negative_image.jpg",
    "roi_image.jpg"
]

existing_files = [f for f in output_files if os.path.exists(f)]

os.makedirs("experiment1_outputs", exist_ok=True)

for file in existing_files:
    shutil.copy(file, "experiment1_outputs")

shutil.make_archive(
    "experiment1_outputs",
    "zip",
    root_dir=".",
    base_dir="experiment1_outputs"
)

print("ZIP file created successfully.")
print("Files included:", existing_files)

ZIP file created successfully.
Files included: ['output_image.jpg', 'output_image.png',
```

```
'grayscale_image.jpg', 'resized_image.jpg', 'rotated_image.jpg',  
'horizontal_flip.jpg', 'vertical_flip.jpg', 'negative_image.jpg',  
'roi_image.jpg']
```